YALNIZ MÜƏLLİM ÜÇÜN · ARXA MONİTOR

Dərs 9 — layihəni addım-addım yaz

Model, migration, ORM, admin
models · migrate · admin · shell
Slaydlar: lesson-09.pdf (40)

Başlanğıc — Dərs 8-in sonundakı layihə. Hər addımda faylın həmin andakı vəziyyəti göstərilir. Yaşıl sətirlər — bu addımda **yeni yazdığın** sətirlərdir; ~~qırmızı~~ — silinən/əvəz olunan sətir; ağ sətirlər artıq fayldadır, `...` — dəyişməyən hissə. Qara bloklar — terminal (çıxışlar real yoxlanılıb). Sağ küncdə — təxmini vaxt və slayd nömrəsi.

0 · Dərsdən əvvəl (evdə, 10 dəq)

1 Hazırlıq

dərsdən əvvəl

1. Dərs 8-in sonundakı layihəni aç (öz `step-blog`-un və ya `project/django/lesson-08/`-in surəti). `blog/data.py` hələ yerindədir — bu gün **silinmir**.

```
cd step-blog
git pull
source .venv/bin/activate
python manage.py runserver     # Dərs 8-in saytı açılmalıdır
```

2. Qovluqda `db.sqlite3` varsa, **sil**. Baza boş başlamalıdır — yoxsa postların id-ləri `data.py`-dakı id-lərlə üst-üstə düşməz (aşağıda addım 8-ə bax).

3. Superuser üçün sadə ad/parol fikirləş (ekranda yazacaqsan): məs. `admin` / `step12345`.

> ⚠ **Ehtiyat plan:** `python manage.py loaddata demo` — `project/django/lesson-09/blog/fixtures/demo.json` (5 post + 3 kateqoriya). Bu fayl **kateqoriyaları da** saxlayır, ona görə yalnız `0002_category` migration-u olandan sonra işləyir. Yəni ancaq praktikadan sonra, öz bazanda. Tələbəyə göstərmə.

I hissə · Slayd + canlı kod (0:00 – 1:30)

2 Giriş və nəzəriyyə — kod yoxdur

0:00–0:20 · slayd 1–11

Slayd 2: keçən dərsdən 5 sual + ev tapşırığına baxış. Slayd 3: “Yeni post yazmaq üçün nə etməliyik?” → `data.py`-ı açıb kod dəyişmək. “Redaktor Python bilmir.” → baza + admin.

Slayd 6–8: baza = Excel cədvəli, SQLite = bir fayl. Lövhədə üç sütun: **class ↔ cədvəl · atribut ↔ sütun · obyekt ↔ sətir**.

> **De:** “Dərs 3-də `Telebe` obyektləri yaddaşda yaşayırdı, proqram bağlananda ölürdü. İndi class eynidir — obyektlər bazada yaşayır.”

Slayd 10–11: `Film` nümunəsi və field tipləri — yalnız slayd, kod yazmırsan.

3 Post modeli

0:20 · slayd 12–15

`blog/models.py`

```
from django.db import models

# Create your models here.
# Model = cədvəl, obyekt = sətir, sahə = sütun.
# Model dəyişəndən sonra HƏMİŞƏ:  makemigrations  →  migrate


# [DƏRS 9] müəllim yazır
class Post(models.Model):
    title = models.CharField(max_length=200)                    # qısa mətn — max_length = maksimum simvol sayı
    author = models.CharField(max_length=50)                    # hələlik ad; Dərs 12-də User olacaq
    category = models.CharField(max_length=50)                  # hələlik slug; Dərs 10-da ForeignKey olacaq
    content = models.TextField()                                # uzun mətn — limitsiz
    is_published = models.BooleanField(default=True)            # dərc olunubmu?
    views = models.PositiveIntegerField(default=0)              # baxış sayı, mənfi ola bilməz
    created_at = models.DateTimeField(auto_now_add=True)        # yaranma vaxtı avtomatik yazılır

    class Meta:
        ordering = ["-created_at"]                              # default sıra: ən yenisi birinci

    def __str__(self):                                          # admin və shell-də belə görünür
        return self.title
```

Sahə-sahə yaz, hər birində dayan. `default` = “dəyər verilməsə nə yazılsın”. `auto_now_add` = “yaranan an özü yazılır, sən toxunmursan”.

⚠ **max_length haqqında:** Django 5.2 + SQLite-da `max_length`-siz `CharField` **xəta vermir** (yoxlanılıb). Ona görə “sil, xətanı göstər” demosunu **etmə**. De: “PostgreSQL/MySQL-da limit lazımdır, real layihədə həmişə yazırıq — başlıq 10 000 simvol olmamalıdır.”

De: “`author` və `category` hələlik adi mətndir — **bilə-bilə** müvəqqəti. Admin-də `oyunlr` yazsan, heç kim saxlamayacaq. Dərs 10-da düzəldəcəyik.” (slayd 15)

De: “Sahə adlarına bax: `title`, `author`, `category`, `content` — `data.py`-dakı açarların eynisi. Bu təsadüf deyil.”

4 makemigrations → migrate

0:40 · slayd 16–19

```
python manage.py makemigrations
Migrations for 'blog':
  blog/migrations/0001_initial.py
    + Create model Post

python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  ...                                  # Django-nun öz migration-ları (cəmi 18)
  Applying blog.0001_initial... OK
  Applying sessions.0001_initial... OK
```

`blog/migrations/0001_initial.py`-ı aç, 10 saniyə göstər:

blog/migrations/0001_initial.py — Django yaratdı, əl ilə dəyişmirik

```
class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel(
            name='Post',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('title', models.CharField(max_length=200)),
                ('author', models.CharField(max_length=50)),
                ('category', models.CharField(max_length=50)),
                ('content', models.TextField()),
                ('is_published', models.BooleanField(default=True)),
                ('views', models.PositiveIntegerField(default=0)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
            ],
            options={
                'ordering': ['-created_at'],
            },
        ),
    ]
```

> **De:** “Migration = bazanın git commit-i. `id`-ni biz yazmadıq — Django özü əlavə etdi. Model dəyişdi → iki komanda. Həmişə.”

`runserver`-dəki “unapplied migrations” qırmızı yazısı artıq yoxdur — göstər.

5 createsuperuser

0:50 · slayd 21

```
python manage.py createsuperuser
Istifadəçi adı (leave blank to use 'radioenerji'): admin
E-poçt ünvanı:                          # boş buraxmaq olar
Password:                               # yazılır, amma ekranda GÖRÜNMÜR
Password (again):
Şifrə çox qısadır. Şifrədə ən az 8 simvol olmalıdır.
Bu şifrə çox asandır.
Bu şifrə ancaq rəqəmlərdən ibarətdir.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

> **De:** “Parol yazanda ekranda heç nə görünmür — bu normaldır, klaviatura işləyir.”

Sadə parolda (`12345`) Django 3 xəbərdarlıq verir → `y` yazıb keç. Güclü parolda (8+ simvol, hərf+rəqəm) bu sual çıxmır. Mətnlərin bir hissəsi ingiliscədir — normaldır.

6 Model-i admin-ə tanıt

0:52 · slayd 21

blog/admin.py

```
from django.contrib import admin

# Register your models here.
from .models import Post

admin.site.register(Post)
```

`/admin/` → daxil ol → **Posts** → “Əlavə et” formasını aç: forma, yoxlama, hamısı hazır.

> ⚠ Formu **göstər, amma SAXLAMA**. İlk post `id=1` -i tutsa, `data.py` -dan köçürəcəyimiz postlar 2–6 olacaq, hələ `data.py` -dan oxuyan kateqoriya/müəllif səhifələrindəki linklər isə **səhv postlara** aparacaq.

7 PostAdmin — siyahını gözəlləşdir

0:55 · slayd 22

blog/admin.py

```
from django.contrib import admin

from .models import Post

admin.site.register(Post)

# [DƏRS 9] müəllim yazır
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ["title", "author", "category", "is_published", "views", "created_at"]
    list_filter = ["is_published", "category"]                  # sağda filtr paneli
    search_fields = ["title", "content"]                        # yuxarıda axtarış qutusu
```

> **De:** “Dərs 4 — `PostAdmin` `ModelAdmin` -dən törəyir, biz yalnız 3 atributu override edirik. `@admin.register` — Dərs 2-dəki decorator.”

> ⚠ `admin.site.register(Post)` sətrini silməyi unudan olacaq → `AlreadyRegistered: The model Post is already registered with 'blog.PostAdmin'.`

Bu dərsdə `is_featured` , `list_editable` , `actions` **yazmırsan** — ev tapşırığıdır.

8 shell — data.py-ı bazaya köçür

1:00 · slayd 24

Slayd 24-ü aç. Shell-də **ilk iş** — `data.py`-dakı 5 postu bir dövrlə bazaya yazırıq. `create()`-i bir cümlə ilə də (“obyekt yarat və bazaya yaz”), təfsilatı növbəti slayddadır:

```
python manage.py shell
>>> from blog.models import Post
>>> from blog.data import POSTS
>>> for p in POSTS:
...     Post.objects.create(title=p["title"], author=p["author"], category=p["category"], content=p["content"])
...
>>> Post.objects.count()
5
```

De: “Dərs 1-dəki `list[dict]` → dövr → hər dict bir sətir oldu. `created_at` vermədik — `auto_now_add` onsuz da qəbul etmir, hamısı indiki vaxtı aldı.”

Yoxla: Admin-də 5 post. `id`-lər 1–5 — `data.py`-dakı ilə eyni. Bu vacibdir: bu gün kateqoriya və ev səhifələri hələ `data.py`-dan oxuyur və `/post/1/` … `/post/5/` linkləri verir.

⚠ Tələbədə id-lər qarışıbsa (importdan əvvəl admin-də post yaradıb): `db.sqlite3`-ü sil → `migrate` → `createsuperuser` → import. 1 dəqiqəlik işdir.

9 create() — tək obyekt

1:03 · slayd 25

```
>>> Post.objects.create(title="Test", author="ali", category="oyunlar", content="Salam")
<Post: Test>                                  # __str__ işlədi
>>> Post.objects.count()
6
```

De: “`views`, `is_published`, `created_at` yazmadıq — default işlədi. `Post.objects` — manager, bütün sorğular ondan başlayır.”

Bu Test postu `id=6` aldı — slayd 28-də onu silirik.

10 Oxumaq — QuerySet

1:05 · slayd 26

```
>>> Post.objects.all()
<QuerySet [<Post: Test>, <Post: FIFA-da ən yaxşı taktika>, <Post: Harri Potter kitablarını hansı ardıcıllıqla oxumalı?>,
<Post: Git nədir və niyə lazımdır?>, <Post: Minecraft-da redstone ilə kalkulyator>, <Post: Python-da ilk addım>]>
>>> Post.objects.filter(author="nigar")
<QuerySet [<Post: Git nədir və niyə lazımdır?>, <Post: Python-da ilk addım>]>
>>> Post.objects.filter(author="nigar").count()
2
>>> Post.objects.exclude(category="oyunlar")
<QuerySet [<Post: Harri Potter kitablarını hansı ardıcıllıqla oxumalı?>, <Post: Git nədir və niyə lazımdır?>, <Post:
Python-da ilk addım>]>
>>> Post.objects.all()[:3]
<QuerySet [<Post: Test>, <Post: FIFA-da ən yaxşı taktika>, <Post: Harri Potter kitablarını hansı ardıcıllıqla oxumalı?>]>
```

De: “Sıra niyə tərsdir? `Meta.ordering = ["-created_at"]` — ən yenisi birinci. Test ən son yarandı.”

11 get() və filter() — fərq

1:08 · slayd 27

```
>>> post = Post.objects.get(id=1)
>>> post.title
'Python-da ilk addım'
>>> Post.objects.get(id=999)
blog.models.Post.DoesNotExist: Post matching query does not exist.
>>> Post.objects.get(author="nigar")
blog.models.Post.MultipleObjectsReturned: get() returned more than one Post -- it returned 2!
>>> Post.objects.filter(id=1).title
AttributeError: 'QuerySet' object has no attribute 'title'
```

> **De:** "`get()` — yalnız unikal sahə ilə (`id`, `slug`). Qalan hər şey — `filter()`."

12 Dəyişmək və silmək

1:10 · slayd 28

```
>>> post = Post.objects.get(id=1)
>>> post.title = "Python-da ilk 10 addım"
>>> post.save()                              # admin-də yenilə — dəyişdi
>>> post = Post.objects.get(id=6)
>>> post.delete()
(1, {'blog.Post': 1})
>>> Post.objects.count()
5
```

`save()`-siz variantı da göstər: başlığı dəyiş, `save()` yazma → admin-də köhnə başlıq.

> ⚠ Başlığı geri qaytar (`post.title = "Python-da ilk addım"` + `save()`) — ya da admin-də düzəlt.

13 Mikro-tapşırıq — shell-də 5 sorğu

1:12 · slayd 29

Tələbələr öz shell-ində yazır (5 dəq). Cavablar səndə:

```
>>> Post.objects.filter(author="murad").count()
2
>>> Post.objects.get(title="Git nədir və niyə lazımdır?").author
'nigar'
>>> Post.objects.exclude(category="oyunlar")           # 3 post
>>> Post.objects.order_by("created_at").first()
<Post: Python-da ilk addım>
>>> post = Post.objects.get(id=2)
>>> post.views = 100
>>> post.save()                                        # admin-də "views" sütununda 100
```

14 View-lar bazadan oxuyur

1:15 · slayd 30–32

`blog/views.py (yalnız post_list və post_detail)`

```
from django.http import Http404
~~from django.shortcuts import render~~
from django.shortcuts import render, get_object_or_404

~~from .data import POSTS, CATEGORIES~~
from .data import POSTS, CATEGORIES                    # hələ lazımdır: kateqoriya və ev view-ları
from .models import Post
...

def post_list(request):
~~    context = {"posts": POSTS}                       # context = adi dict~~
~~    return render(request, "blog/post_list.html", context)~~
    posts = Post.objects.filter(is_published=True)        # SELECT ... WHERE is_published
    return render(request, "blog/post_list.html", {"posts": posts})


def post_detail(request, post_id):
~~    for post in POSTS:~~
~~        if post["id"] == post_id:~~
~~            return render(request, "blog/post_detail.html", {"post": post})~~
~~    raise Http404("Belə post yoxdur")~~
    post = get_object_or_404(Post, id=post_id, is_published=True)   # tapmasa → 404
    return render(request, "blog/post_detail.html", {"post": post})
```

Demo sırası: əvvəlcə `post = Post.objects.get(id=post_id)` yaz → `/post/99/` → 500 (`DoesNotExist`). Sonra `get_object_or_404` → 404. Slayd 31: “Dərs 7-dəki 4 sətirlik dövr — bir sətir.” Sonra admin-də bir postun “is published” işarəsini götür → ana səhifədən itdi. `/post/<id>/` ilə aç → 404. “Filtr detalda da lazımdır” (slayd 32). İşarəni geri qoy.

> ⚠ `from django.http import Http404` və `data.py` importu **qalır** — `category_detail`, `author_posts`, `latest`, `stats` hələ `data.py`-dan oxuyur. Onları tələbə praktikada və evdə köçürəcək.

15 Template-lərə toxunmuruq

1:20 · slayd 33–34

Ana səhifəni, post səhifəsini aç — sayt **eyni** görünür. Template-lərdə bir hərf də dəyişmədi.

> **De:** “Məlumatın mənbəyi dəyişdi — siyahıdan bazaya. Görünüş xəbər tutmadı. MVT budur. Dərs 7-də dict açarlarını model sahələri kimi adlandırmağımızın səbəbi də bu idi.”

Tarix indi `datetime`-dır (bu günün tarixi) — `|date:"d.m.Y"` eyni işləyir.

Slayd 34 (xətalar) — istəsən `no such table`-ı göstər: `db.sqlite3`-ün surətini çıxar, sil, səhifəni aç → `OperationalError: no such table: blog_post`. Faylı geri qoy.

16 Git

1:25

```
git add .
git status           # db.sqlite3 SİYAHIDA OLMAMALIDIR (.gitignore)
git commit -m "Dərs 9: Post modeli, admin, view-lar bazadan"
git push
```

> **De:** “Migration faylları GitHub-a **gedir**, baza (`db.sqlite3`) **getmir**. Dostun clone edib `migrate` edəcək — cədvəllər onda da yaranacaq, amma boş olacaq.”

17 I hissənin sonunda fayllar belə olmalıdır

yoxlama

blog/models.py

```
from django.db import models

# Model = cədvəl, obyekt = sətir, sahə = sütun.
# Model dəyişəndən sonra HƏMİŞƏ:  makemigrations  →  migrate


# [DƏRS 9] müəllim yazır
class Post(models.Model):
    title = models.CharField(max_length=200)                    # qısa mətn — max_length = maksimum simvol sayı
    author = models.CharField(max_length=50)                    # hələlik ad; Dərs 12-də User olacaq
    category = models.CharField(max_length=50)                  # hələlik slug; Dərs 10-da ForeignKey olacaq
    content = models.TextField()                                # uzun mətn — limitsiz
    is_published = models.BooleanField(default=True)            # dərc olunubmu?
    views = models.PositiveIntegerField(default=0)              # baxış sayı, mənfi ola bilməz
    created_at = models.DateTimeField(auto_now_add=True)        # yaranma vaxtı avtomatik yazılır

    class Meta:
        ordering = ["-created_at"]                              # default sıra: ən yenisi birinci

    def __str__(self):                                          # admin və shell-də belə görünür
        return self.title
```

blog/admin.py

```
from django.contrib import admin

from .models import Post


# [DƏRS 9] müəllim yazır
@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ["title", "author", "category", "is_published", "views", "created_at"]
    list_filter = ["is_published", "category"]                  # sağda filtr paneli
    search_fields = ["title", "content"]                        # yuxarıda axtarış qutusu
```

blog/views.py (üst hissə)

```
from django.http import Http404
from django.shortcuts import render, get_object_or_404

from .data import POSTS, CATEGORIES                 # hələ lazımdır: kateqoriya və ev view-ları
from .models import Post

...

def post_list(request):
    posts = Post.objects.filter(is_published=True)        # SELECT ... WHERE is_published
    return render(request, "blog/post_list.html", {"posts": posts})


def post_detail(request, post_id):
    post = get_object_or_404(Post, id=post_id, is_published=True)   # tapmasa → 404
    return render(request, "blog/post_detail.html", {"post": post})

# about, category_list, category_detail, contact, author_posts, latest, stats — DƏYİŞMƏYİB (data.py-dan)
```

Yoxla: `blog/migrations/` -da yalnız `0001_initial.py` . Sayt işləyir: `/` 5 kart, `/post/3/` “Git…”, `/category/oyunlar/` (hələ data.py) → linklər `/post/2/` , `/post/5/` düzgün postlara aparır.

II hissə · Praktika — tələbə kateqoriyaları bazaya köçürür (1:40 – 2:50)

Yeni heç nə izah etmirsən. Tələbəyə de: “ `Post` üçün nə etdiksə, eynisi. Öz `models.py` və `admin.py` -na bax.” Hər blokun sonunda 2 dəqiqəlik checkpoint.

Blok	Tələbə nə edir	Checkpoint — ekranda yoxla
A · 22 dəq slayd 37	`Category` modeli (`name` , `slug` unique, `description` blank) + `Meta` + `__str__` + `makemigrations` / `migrate`	`0002_category.py` yarandı, `migrate` OK
B · 22 dəq slayd 38	`CategoryAdmin` (`list_display` , `prepopulated_fields`) + shell-də `CATEGORIES` import	admin-də “Categories”, 3 kateqoriya adı ilə; yenisini yazanda slug özü dolur
C · 22 dəq slayd 39	`category_list` / `category_detail` ORM ilə + `get_object_or_404` + push	`/category/oyunlar/` 2 kart · `/category/futbol/` 404

18 Praktikanın həlli — yoxlamaq üçün

tələbəyə göstərmə

blog/models.py (Post-un altına)

```
# [PRAKTİKA 9] tələbə yazır
class Category(models.Model):
    name = models.CharField(max_length=100)
    slug = models.SlugField(unique=True)                    # URL-də işlənir, təkrarlana bilməz
    description = models.TextField(blank=True)              # blank=True → formda boş qala bilər

    class Meta:
        ordering = ["name"]
        verbose_name_plural = "Categories"                  # admində "Categorys" yazılmasın

    def __str__(self):
        return self.name
```

```
python manage.py makemigrations
Migrations for 'blog':
  blog/migrations/0002_category.py
    + Create model Category
python manage.py migrate
```

blog/admin.py

```
~~from .models import Post~~
from .models import Post, Category
...

# [PRAKTİKA 9] tələbə yazır
@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ["name", "slug"]
    prepopulated_fields = {"slug": ["name"]}                # ad yazdıqca slug özü dolur
```

```
>>> from blog.models import Category
>>> from blog.data import CATEGORIES
>>> for c in CATEGORIES:
...     Category.objects.create(name=c["name"], slug=c["slug"], description=c["description"])
...
>>> Category.objects.count()
3
```

blog/views.py

```
~~from .models import Post~~
from .models import Post, Category
...

# ──────────────────────── [PRAKTİKA 9] ────────────────────────

def category_list(request):
    categories = Category.objects.all()
    return render(request, "blog/category_list.html", {"categories": categories})
```

```python
def category_detail(request, slug):
    category = get_object_or_404(Category, slug=slug)
    posts = Post.objects.filter(category=slug, is_published=True)   # Dərs 10-da bu sətir sadələşəcək
    context = {"category": category, "posts": posts}
    return render(request, "blog/category_detail.html", context)
```

Praktikada görəcəyin xəta	Səbəb
`no such table: blog_category`	`makemigrations` + `migrate` edilməyib
`No changes detected`	`models.py` yadda saxlanmayıb
`UNIQUE constraint failed: blog_category.slug`	shell importu iki dəfə işlədilib → `Category.objects.all().delete()`, yenidən
Admin-də `Categorys`	`verbose_name_plural` yazılmayıb
Admin-də `Category object (1)`	`__str__` yoxdur
Admin-də `Category` görünmür	`@admin.register` yoxdur və ya `from .models import Category` unudulub
`NameError: name 'Category' is not defined`	`views.py`-da import yenilənməyib
`Category matching query does not exist` (500)	view-da `get()` — `get_object_or_404` işlət
`/category/oyunlar/` boşdur	`filter(category=category)` — obyekt verilib, slug lazımdır: `category=slug`

Son 10 dəqiqə (2:50 – 3:00)

19 Yekun və ev tapşırığı

slayd 40

Slayd 40-ı birlikdə oxu, `hw-09.pdf`-i payla. Push: `git add . → git commit -m "Dərs 9: praktika" → git push`.

Dərs 10-a körpü: admin-də bir postun kateqoriyasını `oyunlr` yaz → kartdakı link `/category/oyunlr/` → 404. Soruş: “Kateqoriyanın adını dəyişsək, postlarda nə olacaq?” → Dərs 10: `ForeignKey`.

20 Ev tapşırığının həlli

yoxlamaq üçün

Tam həll bu qovluqdadır — `[EV 9]` nişanlı hissələr. Əsas parçalar:

`blog/views.py`

```python
# tapşırıq 1 — post_detail-də, render-dən əvvəl
post.views += 1
post.save()

# tapşırıq 2
posts = Post.objects.filter(author=name, is_published=True)

# tapşırıq 3
posts = Post.objects.filter(is_published=True).order_by("-created_at")[:3]

# tapşırıq 4 — stats
published = Post.objects.filter(is_published=True)
context = {
    "post_count": published.count(),                          # SELECT COUNT(*) — siyahını yükləmir
    "category_count": Category.objects.count(),
```

```
    "top_post": published.order_by("-views").first(),        # ən çox baxılan; yoxdursa None
}
```

blog/models.py (Post, created_at-dan sonra)

```
    # [EV 9] tapşırıq 5 — yeni sahə → yeni migration
    is_featured = models.BooleanField(default=False)
```

```
python manage.py makemigrations
Migrations for 'blog':
  blog/migrations/0003_post_is_featured.py
    + Add field is_featured to post
```

blog/admin.py — tapşırıq 6

```
    list_display = ["title", "author", "category", "is_published", "is_featured", "views"]
    list_filter = ["is_published", "is_featured", "category"]
    list_editable = ["is_published", "is_featured"]
```

> ⚠ `list_editable`-dakı sahə `list_display`-də yoxdursa: `(admin.E122) The value of 'list_editable[1]' refers to 'is_featured', which is not contained in 'list_display'.`

Tapşırıq 7: `blog/data.py` silinir, `views.py`-dan `data` importu və `Http404` çıxır. Yoxlama: bütün səhifələr açılır, `ImportError` yoxdur. Migration tarixçəsi: `0001_initial` → `0002_category` → `0003_post_is_featured`.